

Challenges in LLMs training

Haitian Sun - 03/2025

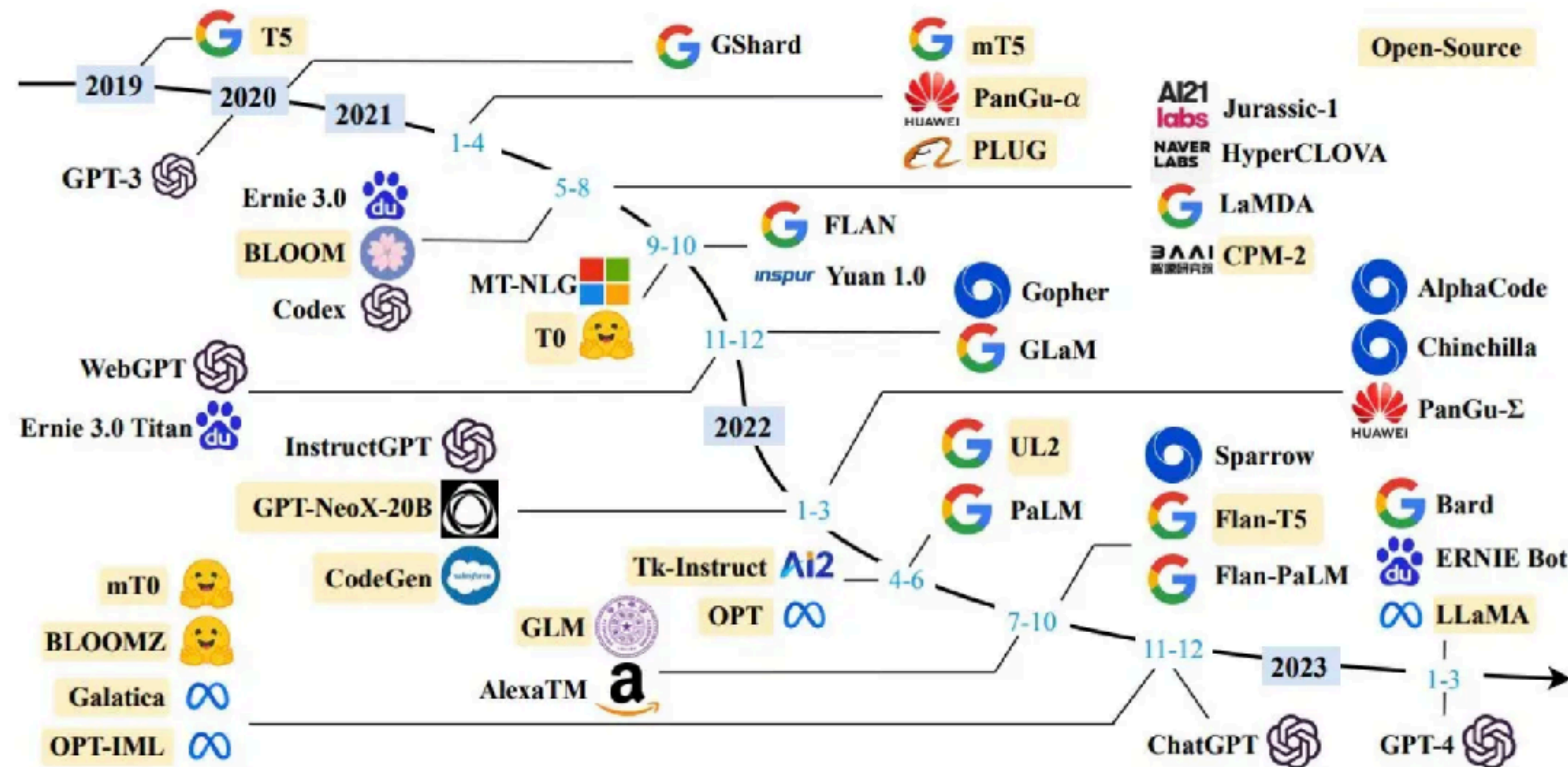
Research Scientist at Apple, Ex-Google Deepmind

Background

- Where are LLMs from?
 - Prior to 2018, researchers (academic and industry) build specialized NN models for various NLP tasks, e.g. POS tagging, NER, QA, ...
 - BERT and GPT-2: models with 100M ~ 1B parameters
 - Multi-tasking: one model, many tasks
 - Larger models: T5, FLAN, GPT-3, GPT-4, Llama...
 - Scaling law: More parameters, more data, better performance

Scaling Law

- More parameters, more training data, better performance [From publicly released information...]
- Gemini Ultra, GPT-4 (estimated): > 1T parameters
- Llama 3: Upto 15 trillion tokens



References:

<https://www.labellerr.com/blog/challenges-in-development-of-llms/>

<https://www.linkedin.com/pulse/gemini-googles-multimodal-mastermind-its-impact-ai-manish-tyagi-du2tf>

<https://ai.meta.com/blog/meta-llama-3/#:~:text=To train our largest Llama,detection, handling, and maintenance.>

Scaling

- One machine is not enough...
 - We can't load all data into CPU memory (trillions of tokens, i.e. TBs)
 - We can't do all computation on 1 GPU (~10 billions of parameters)
- How do we deal with these challenges?

Today

- How to scale LLMs?
 - Data
 - Computation
 - Operation

Scaling datasets

- Datasets from many sources
 - Books
 - Journals
 - Web forums
 - Blogs
 - Ads
 - PDFs
- We will randomly shuffle and mix them - randomness is #1 in training LLMs

Scaling datasets - local buffer

- Challenge: We can't load everything into memory to shuffle
- Option 1: Shuffle locally with a large buffer, e.g. `buffer_size=65536`
 - Streaming data from the start of a dataset
 - Load the first ``buffer_size`` examples into memory
 - Randomly sample a batch (e.g. 1024 examples) from the buffer
- ****Assumption****: With a “large enough” buffer, we get sufficient randomness
- Usually works well with post-training / supervised fine-tuning (SFT), but not enough for pre-training
 - Some domains have many many similar documents - NOT random!!!

Scaling datasets - index & random access

- Challenge: We can't load everything into memory and shuffle
- Option 2: Index the dataset and then access with index, e.g. grain [\[link\]](#)
 - Go through the dataset once to build an index
 - Sample a `batch\_size` of random ids, and then load data at the indices
- It may have some overhead reading from the file system, but it can be well optimized - interleaving with model training
- Optionally, we can generate a deterministic dataset offline as part of data pre-processing

Scaling datasets - deterministic datasets

- Globally shuffle on disk
- Write randomly shuffled data into a file, i.e. a deterministic dataset
- Sometimes lack of flexibility
 - e.g. if we would like to change mixture weight...

Today

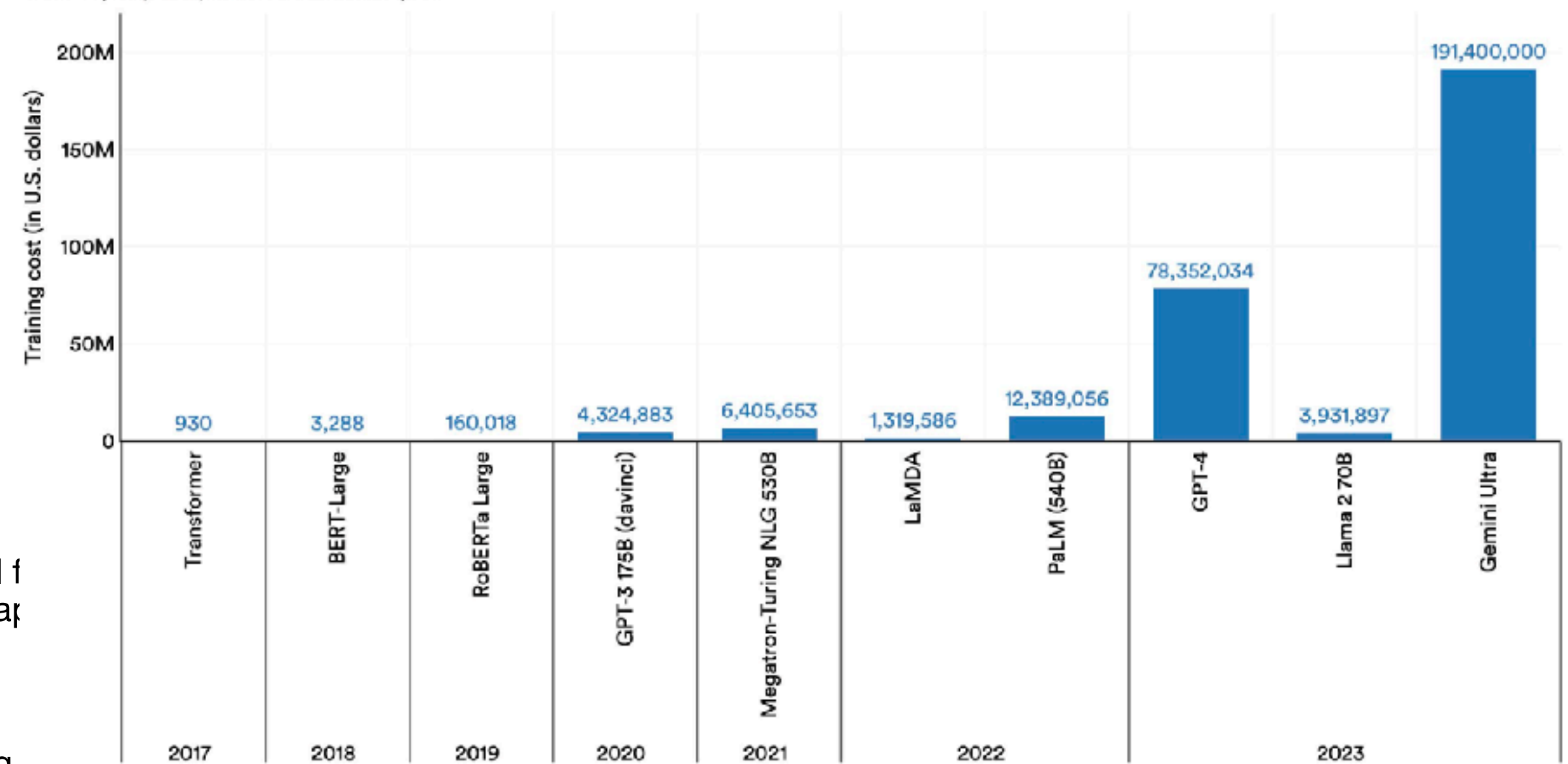
- How to scale LLMs?
 - Data
 - Computation
 - Operation

Scaling computation

- Multi-GPUs
 - Compute: Llama 3 uses “two custom-built 24K GPU clusters”.
 - Nvidia H100: \$30,000 each
- Very expensive...

Estimated training cost of select AI models, 2017–23

Source: Epoch, 2023 | Chart: 2024 AI Index report



By Stanford Institute for Human-Centered Artificial Intelligence (permission obtained by email from the AI index research manager) - <https://aiindex.stanford.edu/report/#individual-chapters> (chapter image 3), CC BY-SA 4.0, <https://commons.wikimedia.org/w/index.php?curid=149139090>

<https://ai.meta.com/blog/meta-llama-3/#:~:text=To train our largest Llama,detection, handling, maintenance.>

Scaling computation - Parallelisms

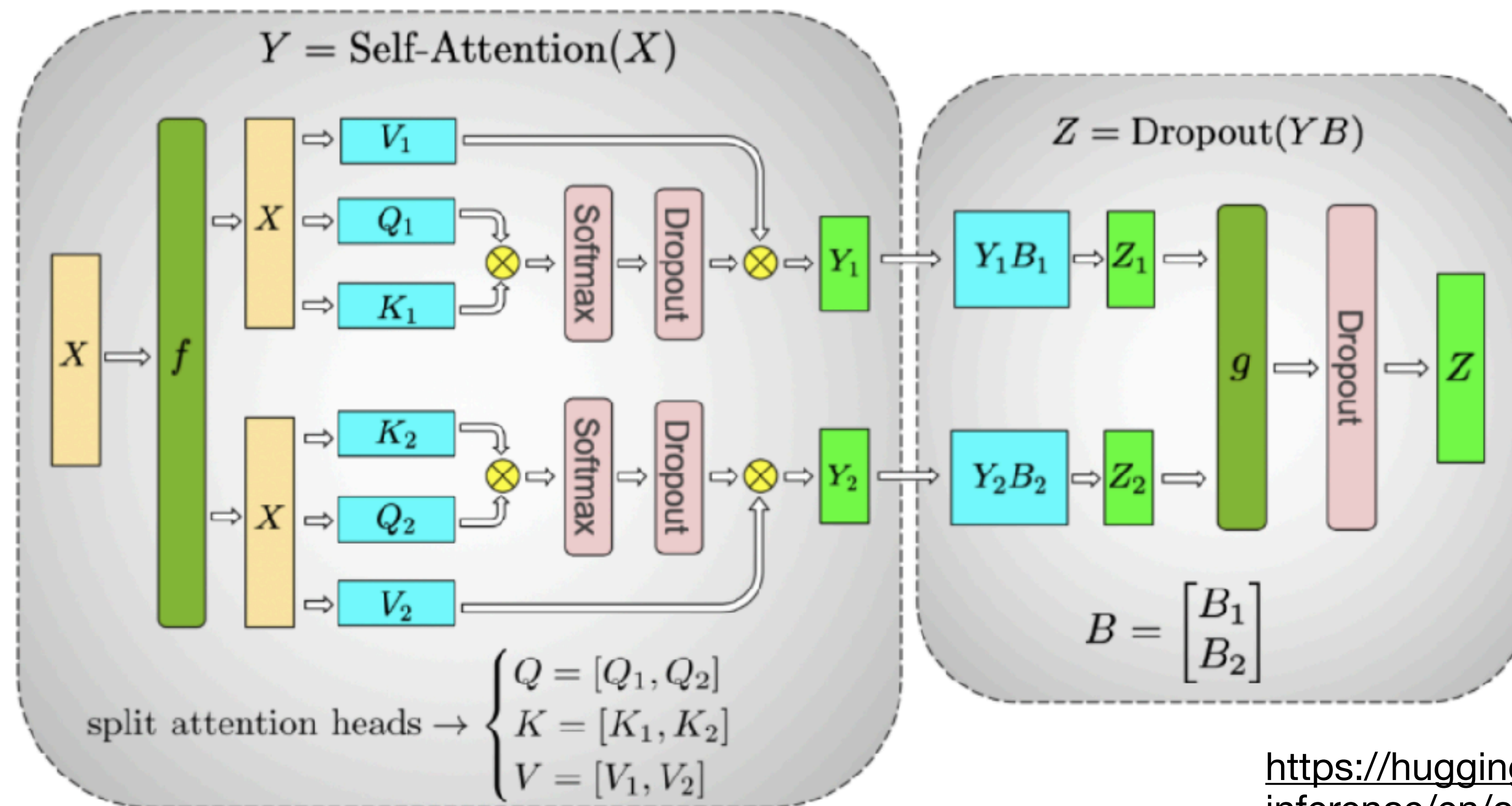
- Why? We don't have large enough GPU memory
 - We need 1000x GPUs work together
- Commonly used parallelisms
 - Data parallelism
 - Tensor parallelism
 - Pipeline parallelism
 - Context parallelism

Data parallelism

- This is the simplest parallelism :)
- Split a big training batch into smaller ones and send to different GPUs
 - E.g. if we have 4 GPUs and a batch of 8 examples
 - Example 1 & 2 -> GPU 1
 - Example 3 & 4 -> GPU 2
 - Example 5 & 6 -> GPU 3
 - Example 7 & 8 -> GPU 4
 - Sync gradients afterward

Tensor Parallelism

- Split large matrix into smaller ones
 - E.g. multi-head attention: split attention heads onto different GPUs

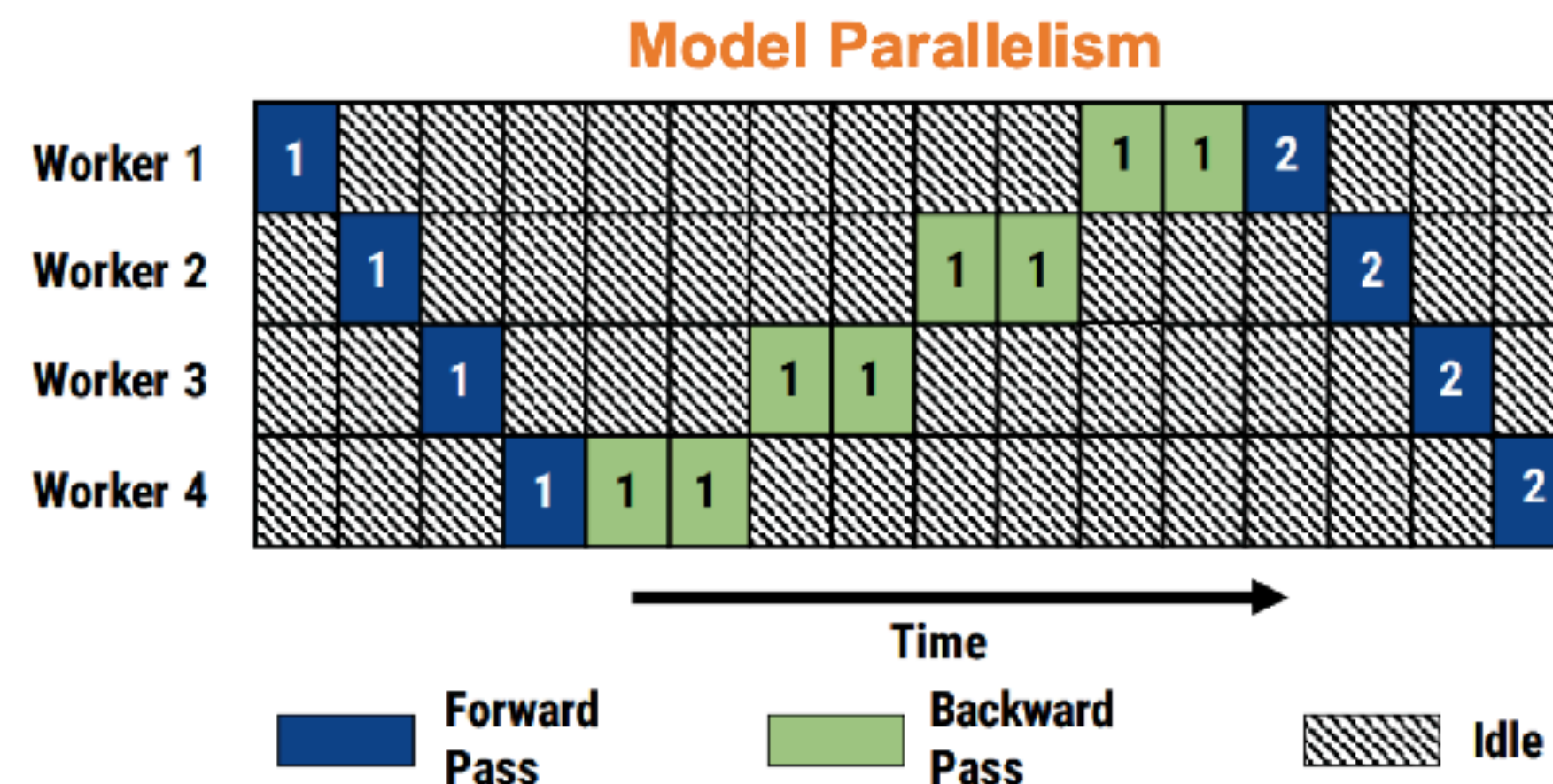


https://huggingface.co/docs/text-generation-inference/en/conceptual/tensor_parallelism

<https://arxiv.org/abs/1909.08053>

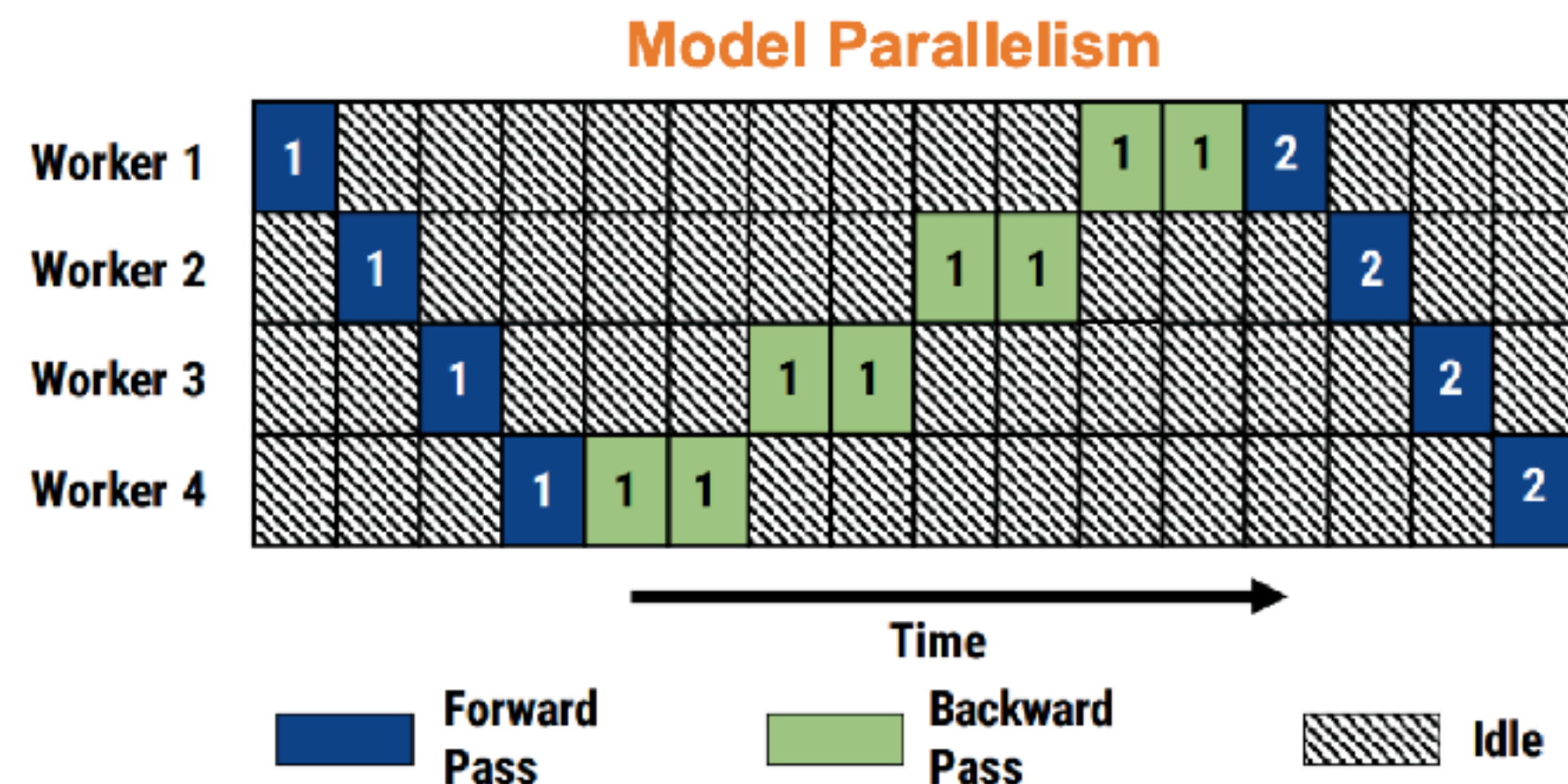
Pipeline Parallelisms

- To run large models, we divide big models into stages
- In Transformer, we can split by Transformer layers
 - GPU #1: layer 1-16, GPU #2: layer 17-32, GPU #3: layer 33-48, GPU #4: layer 49-64
- Sequentially execute the layers, i.e. GPU #1 -> #2 -> #3 -> #4 (forward)
- Waste resources, e.g. GPU #1, #3, #4 are idle while #2 is running



Pipeline Parallelisms

- Pipeline parallelism: run multiple stages in parallel
- Example:
 - Timestamp 1: GPU #1 on mini-batch @1
 - Timestamp 2: GPU #1 on mini-batch @2, GPU #2 on mini-batch @1
 - Timestamp 3: GPU #1 on mini-batch @3, GPU #2 on mini-batch @2, GPU #3 on mini-batch @1
 - ...
- Significantly reduce the expected idle time, i.e. grey area is smaller



Context Parallelisms

- This is still a hot research area...
- General idea: Split long context into **blocks**, compute each block locally, then sync globally
 - E.g. for a context with 1M tokens, split into blocks with 4096 tokens -> 256 blocks
 - Each GPU works on one block (i) of tokens to compute attention (Q_i , K_i , V_i)
- However, we need to attend to the full context, not only tokens in block (i)
 - Communicate with other GPUs to get K_j 's and V_j 's to compute attention (Q_i , K , V)
 - Slow and expensive...
- Many other tricks to improve efficiency
 - Liu and Abbeel - "Blockwise Parallel Transformer for Large Context Models"
 - Jacobs et al - "DeepSpeed Ulysses"

Scaling computation - Parallelisms

- How to determine the optimal parallelisms (i.e. shardings)?
 - $\# \text{ GPUs} = \text{data} * \text{tensor} * \text{pipeline} * \text{context}$
 - E.g. $2048 \text{ GPUs} = 8 (\text{data}) * 8 (\text{tensor}) * 8 (\text{pipeline}) * 4 (\text{context})$
 - Alternatively, $2048 \text{ GPUs} = 4 (\text{data}) * 1 (\text{tensor}) * 64 (\text{pipeline}) * 8 (\text{context})$
- How do I decide the size of each dimension?
 - Depending on model architecture, data format, GPU performance, network, and many other factors, there's no answer until...
 - Experiments!

Today

- How to scale LLMs?
 - Data
 - Computation
 - Operation

Everything can go wrong...

- Compute
 - Numerical stability - so many computations (~100 teraFLOPs) happening at the same time, almost impossible to detect...
 - $1 + -1 = 0.0001$
 - $0 + 0 = \text{very_negative_number}$
- Solutions?
 - Not too much we can do...
 - Kill the job and restart from a previous healthy checkpoints.

Everything can go wrong...

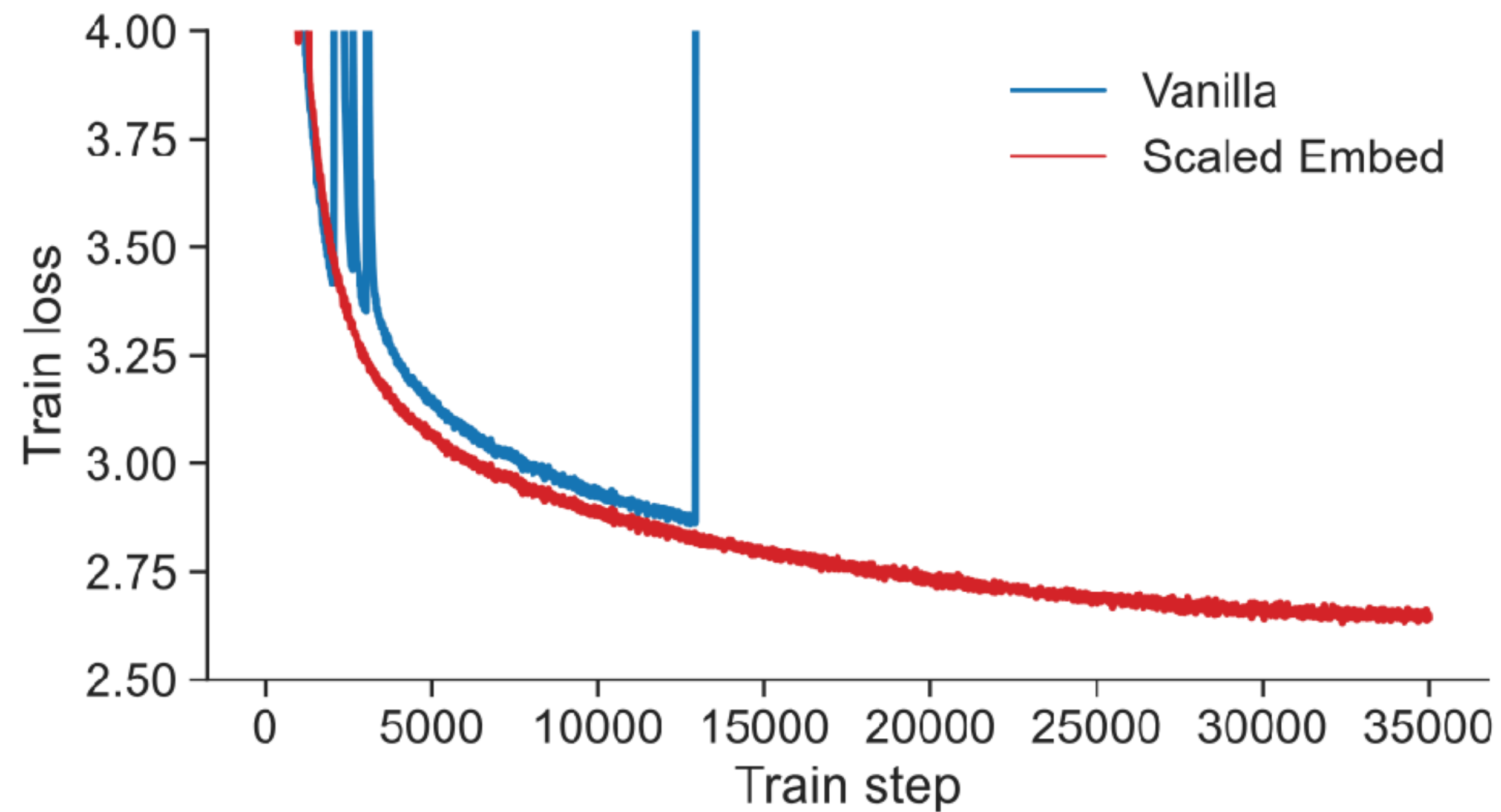
- Data
 - Web data are very noisy - random strings, characters
 - Loss and gradient will likely go wrong (causing spikes)
 - Solution: build filters to remove bad data before training
 - May also help: different optimizers / hyper-parameters / gradient clipping
- If nothing works, restart from a healthy checkpoints...

Everything can go wrong...

- Clusters can go down during training for many reasons
 - Out of power, overheating, resource management (e.g. kubernetes) bugs, network issues...
- Solutions?
 - **Checkpoint everything!!!** Make sure you can recover.
 - Model & optimizer parameters
 - dataset iterators

Everything can go wrong...

- As a result...

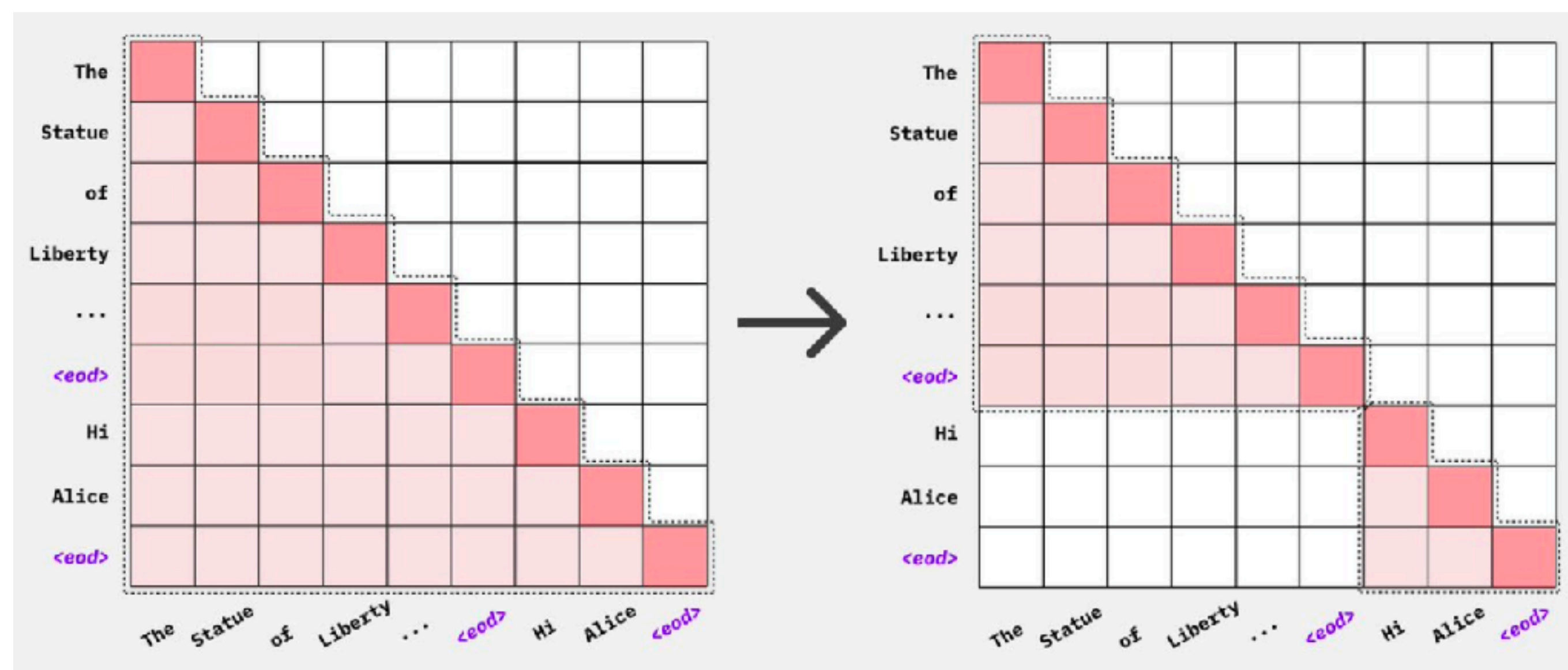


General suggestions in ops

- # 1 recommendation: Always checkpoint your training. Make sure everything is recoverable!!!
- Keep your clusters for data and checkpoint closer to your model
- Automatically monitor your job, so you can restart it when things go wrong (not wasting GPU time)
- A lot more...

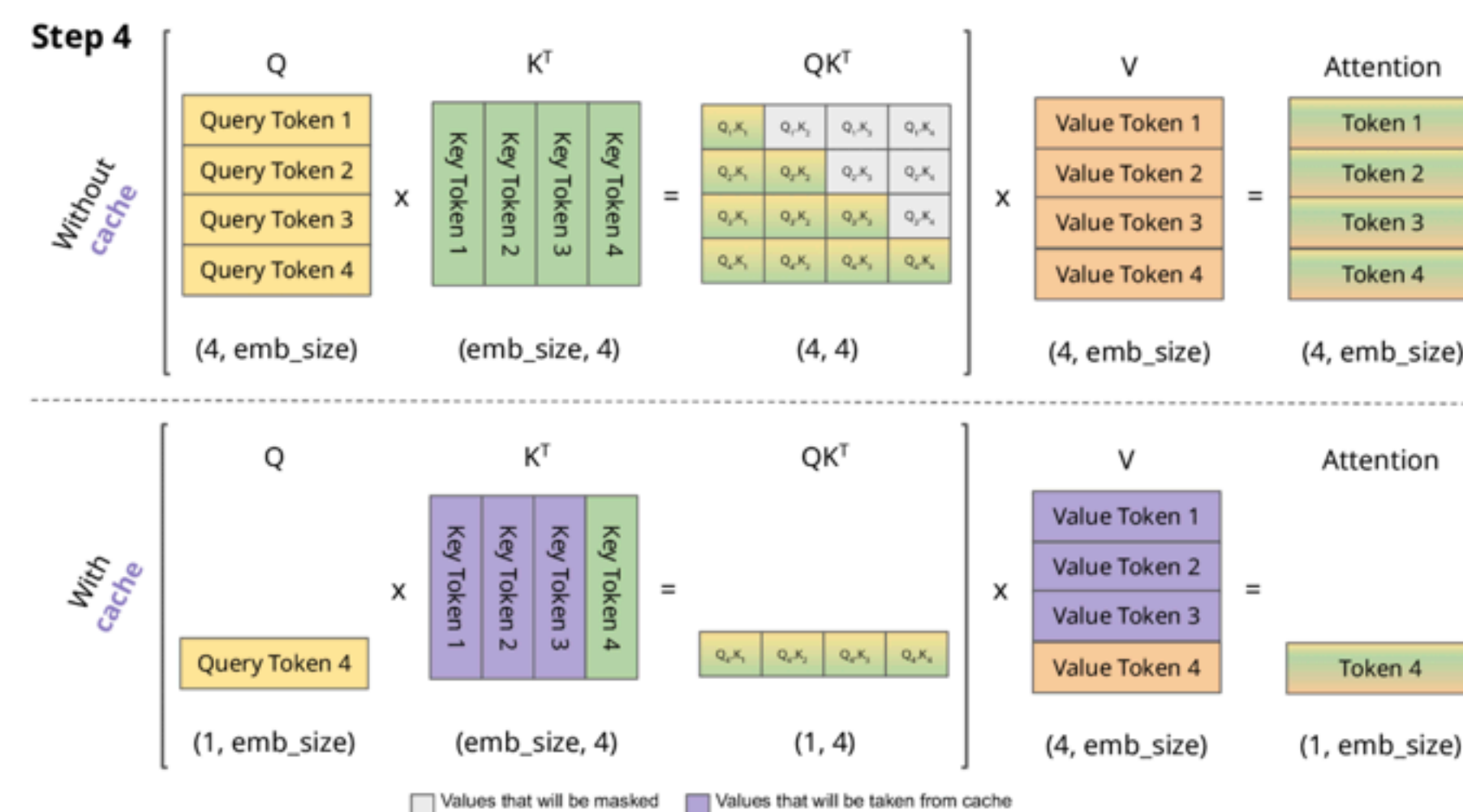
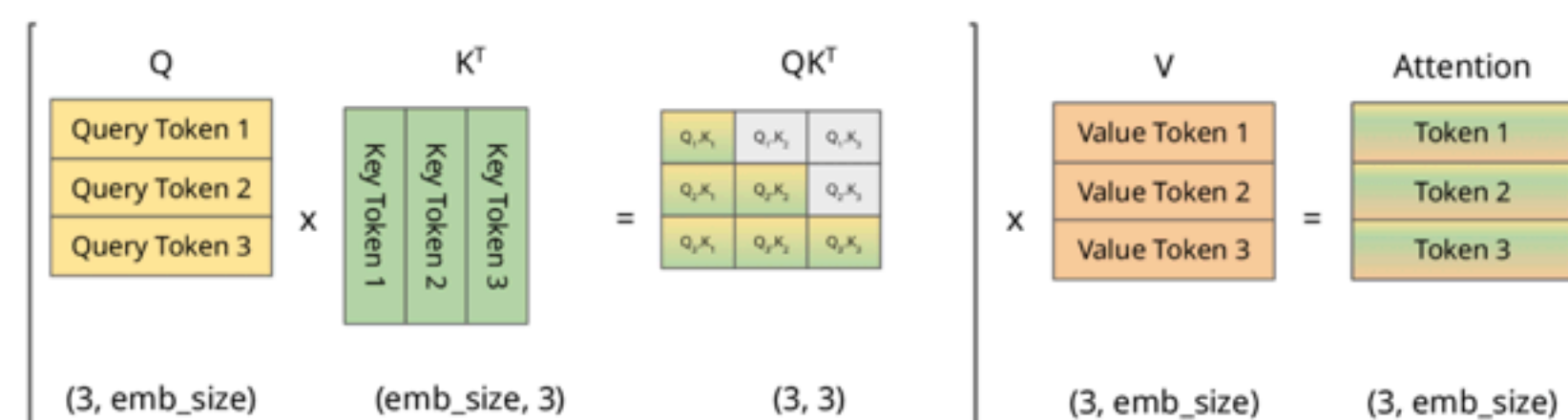
Other tricks: Packing

- Save memory by re-use padded tokens!!!
 - To accomodate long inputs, we need to increase context length, e.g. 8192
 - But for short inputs, e.g. with 200 tokens, we waste $8192 - 200 = 7992$ context length...
- Combine multiple examples into one. Use input masks separate them.
 - Example: “The statue of Liberty ...” and “Hi Alice”



Other tricks: KV Caching

- In decoding, to predict the n 'th token, we need the embeddings of the previous $n-1$ tokens...
- Recompute all $n-1$ tokens' embeddings? Very expensive!!!
- Cache the embedding of token 1, ..., $n-1$ once computed, so we can reuse them to compute the n 'th token



Other tricks (will not cover)

- Flash attention
- Speculative decoding
- ...

Thank you!